

Project Overview: Graph Theory & Field Routing for Paragominas

1. Calculate Travel-Time Weights (The "Cost" Function)

- **What we do:** Compute the physical length of each road segment in meters, assign an estimated speed (km/h) based on its highway classification, and derive total travel time in hours.
- **Why it matters:** Field inspectors need to know the *fastest* route, not just the physically shortest one. A rough dirt track or unclassified trail carries a heavy penalty (low speed) compared to a paved highway, ensuring realistic travel estimates.

2. Spatial Snapping & Topology Correction (Connecting the AI Traces)

- **What we do:** Build an R-tree spatial index on the official OSM road backbone and iterate through PrevisIA AI logging traces, snapping their endpoints to nearby official infrastructure using a controlled distance threshold (e.g., 15 meters) and `shapely.ops.snap`.
- **Why it matters:** AI-derived satellite traces often float a few meters away from official roads. Without topological snapping, they form isolated digital islands, and a router cannot find a path from a town onto a hidden logging trail.

3. Dual-System Exploratory Visualization & Reclassification

- **What we do:** Transform coordinates from native WGS84 to local metric units (UTM Zone 22S - EPSG:31982), group granular OSM and AI tags into three clean operational groups (*Paved / Main Roads*, *Rural / Dirt Tracks*, and *AI Traces*), and export WGS84 files for visual auditing in Google Earth Pro.
- **Why it matters:** Raw datasets contain dozens of minor tags and misaligned coordinate vertices. Simplifying the network and visualizing it in 3D ensures data integrity and provides clear, readable maps for remote field operations.

4. Converting LineStrings to NetworkX Nodes and Edges

- **What we do:** Parse GeoDataFrame geometries so that every road intersection and endpoint becomes a NetworkX tuple node `(x, y)`, and every road segment becomes a weighted edge carrying length metrics and travel-time weights.
- **Why it matters:** NetworkX cannot route directly over raw GeoPandas MultiLineStrings; it requires a mathematical node-and-edge graph structure to perform network analysis.

5. Largest Connected Component Extraction & Cleaning

- **What we do:** Filter the graph using `nx.connected_components` to isolate and retain only the largest fully interconnected network core, discarding disconnected forest stubs, floating fragments, and dead ends.
- **Why it matters:** Real-world spatial data is riddled with fragmentation. Extracting the main component ensures that routing queries only operate on a robust, traversable network without crashing due to unreachable paths.

6. Running the Shortest Path Query & Field Export

- **What we do:** Implement a robust routing function using SciPy `KDTree` spatial lookups to snap input GPS coordinates to the nearest clean graph nodes, running Dijkstra's algorithm to compute the optimal route and exporting turn-by-turn GeoJSON files for field apps.
- **Why it matters:** This yields the final, reliable turn-by-turn navigation and map layers that Amazon field inspectors can deploy directly into field mapping apps to reach otherwise unroutable, unmapped destinations.

Conclusions:

Mapping our final layers on Android was easy.

Mapping them on my iPad changed my symbology and stopped me from mapping both the raster Sentinel-2 high-resolution layer with my road networks.

95% of roads mapped are disjoint sets in graph theory, or unconnected.

This corroborates what we know about the disjointed data provided and that many Amazon roads are disconnected for nefarious purposes like mining, deforestation.

Utilize the QField Package to edit these cleaned layers as more roads become connected.

Utilize field worker insights and drone data collections methods to continue optimizing the field app.